COMP532 - Machine Learning and Bio-inspired Algorithms: Assignment I
Bandit Arm Problem

March 7, 2026

University of Liverpool
Zhiheng Wang - Z.Wang252@liverpool.ac.uk - 201865071

Laura Sierra Pena - l.v.sierra-pena@liverpool.ac.uk - 201936067
Ainur Smailova - Sgasmai2@liverpool.ac.uk - 201874285
Yan-Yu Lin - sgylin20@liverpoll.ac.uk - 201948160
Joe Davies - J.Davies51@liverpool.ac.uk - 201950629
Pablo Ramos - sgpramos@liverpool.ac.uk - 201885602

Introduction

Reinforcement learning is an artificial intelligence approach that uses agents set to take actions in complex environments and learn from their interactions. The objective is to maximise the expected cumulative reward.

As defined by Sutton & Barto, 2018, Ch.2, Sec 2.1, Multi-armed bandits (MABs) are a simplified reinforcement learning setting where an agent repeatedly chooses among several bandits to maximise reward under uncertainty.

A multi-armed bandit problems can be interpreted as a casino parlor: multiple slot machines, which are referred to as bandits, are set to give random rewards. The value of these rewards and their likelihood can be hardcoded to behave like a probability density function. I.E. the slot machines, when pulled, may give positive or negative rewards that converge around a mean. That said, many kinds of statistical distributions may be implemented, such as uniform distributions with upper and lower bounds, triangular distributions with a mode, and so on.

Using reinforcement learning, agents must be set with some kind of policy that enables taking the optimal choice among the multiple slot machines, I.E. the bandits, available. Sequentially pulling on different bandits, will influence the agents' decisions by providing evaluative feedback.

A central difficulty is the exploration exploitation trade-off. When action value estimates are maintained, the arm with the highest current estimate is the greedy choice. Selecting it is exploitation because it uses the best option according to current knowledge, while selecting a non-greedy arm is exploration because it can reduce uncertainty and improve future estimates. Exploitation tends to maximise expected reward in the next step, but exploration can lead to higher total reward over a longer horizon by helping the agent discover arms that are actually better. Since a single action choice cannot do both at once, balancing exploration and exploitation is intrinsical in bandit problems.

A common way to implement action value methods is to estimate each arm's value from observed rewards. One simple estimator is the sample average, computed as the total reward

collected from an arm divided by the number of times it has been selected. (Antonacopoulos et al., 2025)

To compare bandit methods, performance is often visualised using reward based curves such as cumulative or average reward over time, together with the frequency of selecting the optimal arm. (Antonacopoulos et al., 2025)

In many real applications, reward distributions are non-stationary and can change over time. This motivates methods that adapt to recent observations rather than relying on full historical averages. Recent work has also explored explicit uncertainty measures, for example using the fractal dimension of reward history and integrating it into standard bandit algorithms, with reported improvements in cumulative reward in non-stationary settings. (Nandhini & Pandian, 2025)

Methodology

Table 1: Agents and Environments

Estimator	Strategy	Environment
Total Average	epsilon = 0.1	Static (Stationary Bandits)
Total Average	epsilon = 0.3	Static (Stationary Bandits)
Total Average	epsilon = 0.5	Static (Stationary Bandits)
Total Average	epsilon = 0.7	Static (Stationary Bandits)
Total Average	epsilon = 0.9	Static (Stationary Bandits)
Total Average	Softmax	Static (Stationary Bandits)
Total Average	Softmax	Dynamic (Non-stationary Bandits)
Moving Average (window=100)	Softmax	Dynamic (Non-stationary Bandits)
Moving Average (window=20)	Softmax	Dynamic (Non-stationary Bandits)
Inc Update (step = 0.1)	Softmax	Dynamic (Non-stationary Bandits)
Epsilon-Greedy (epsilon=0.1, step=0.1)	Softmax	Dynamic (Non-stationary Bandits)
Inc Update (step = 0.3)	Softmax	Dynamic (Non-stationary Bandits)

To approach the problem, a set of agents was formulated. In Table 1 the designed agents can be seen in terms of their estimator, strategy (I.E. its policy) and which environment it faced. The two environments were designed as follows:

- A static environment consisting of many bandits behaving according to a mixture of statistical distributions with fixed parameters. This scenario tests the policies Softmax and Espislon-Greedy against each other. Fixed distribution bandits are ideally estimated with average estimators, which are set equally to all the agents.

- A dynamic environment consisting of the same bandits, this time with changing parameters. Given that the best performing policy in the previous scenario was Softmax, it was assigned to most of the agents, and different estimators were tested. A single Espislon-Greedy policy agent was kept as reference.

After drafting different programming approaches, full object-oriented programming was chosen because it presented an efficient and effective way to implement the multi-armed bandit algorithm with multiple configurations:

- The base abstract arm class was created with multiple subclasses, each with different distributions and the methods required to update and reset them, refer to Figure 1.

- An Estimator class allows building the agents with different kinds of estimators refer to Figure 2.
- The Strategy class, which selects which action to perform based on the estimations, and finally an Agent class, which is the combination of an estimator and a strategy, as well as recording the cumulative reward over time, refer to Figure 3.

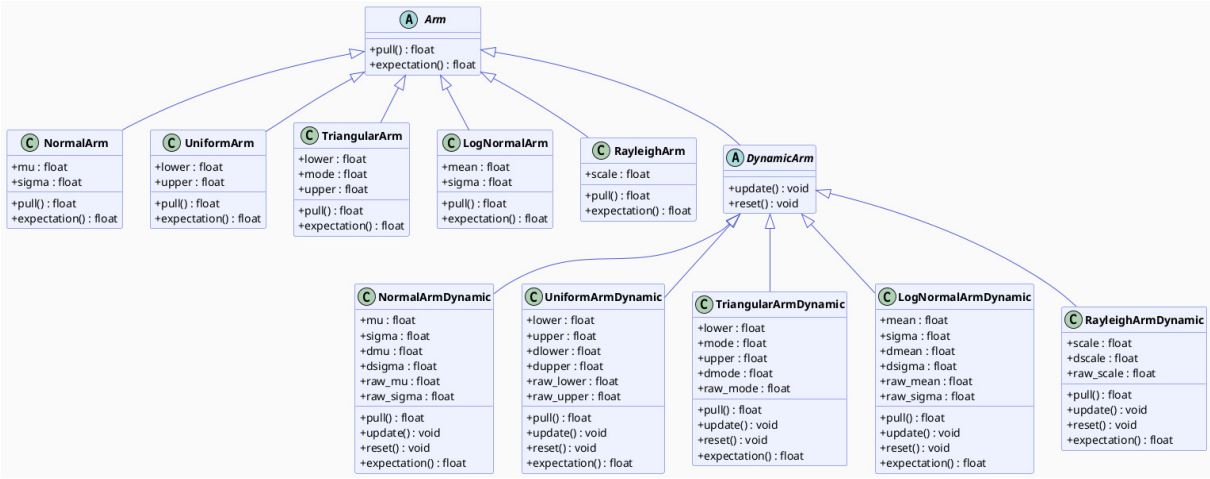

Figure 1: Design of the Bandit class.

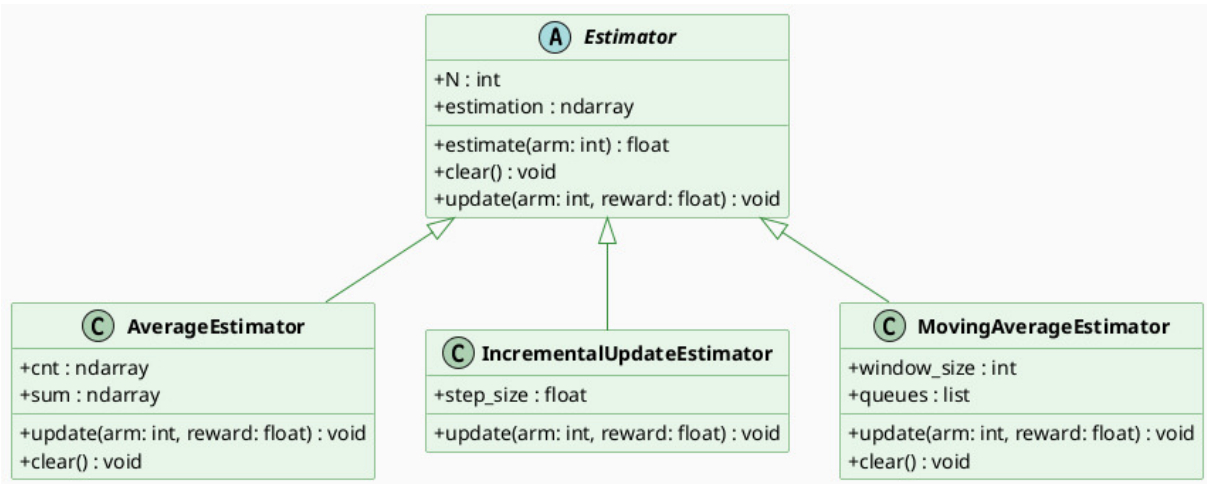

Figure 2: Design of the Estimator class

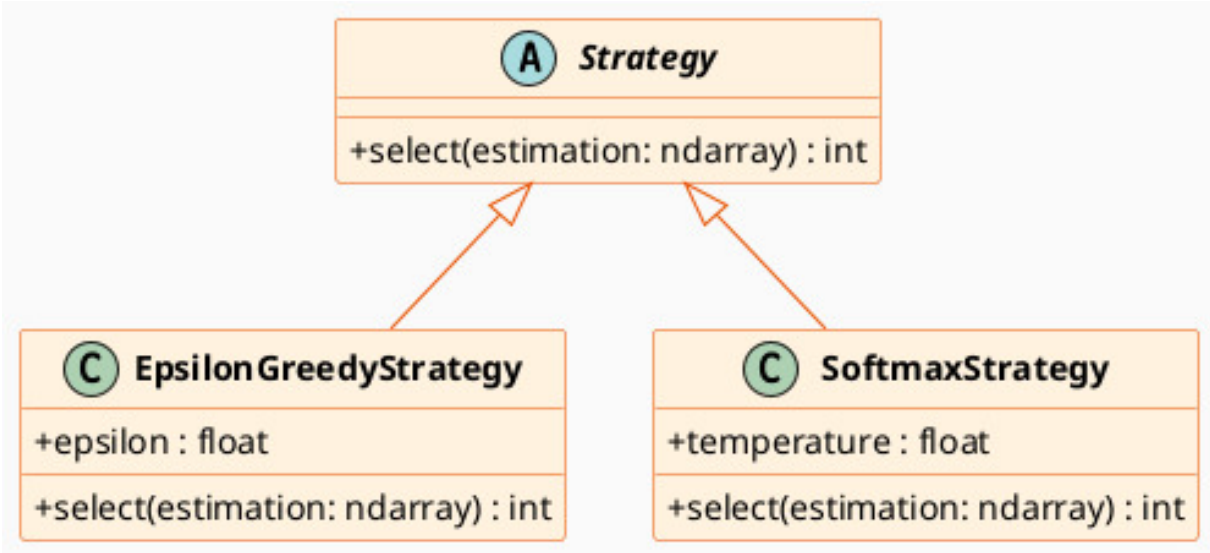

Figure 3: Design of the Strategy class

In each step, each agent selects an arm using its specific strategy. The agent receives a reward from the selected arm, and the agent updates its estimator with that reward. After that, the results are then plotted with cumulative reward in the y-axis and timesteps in the x-axis. The results show a range of cumulative rewards over timesteps, where a lower epsilon value corresponds to a higher cumulative reward.

To ensure reproducibility in between runs, we used NumPy's random seed module and selected different values for the seeds.

Results

0.1 Static Environment

In Figure 4 it can be observed that over many steps, small values for ε and Softmax strategies accumulate reward much faster, leading to clearly higher curves. Strategies with large ε show significantly lower cumulative reward.

The patterns match the best-arm plot: small ε (0.1, 0.3) and Softmax achieve higher steady-state average reward. Large ε values suffer from excessive exploration, so their average reward stabilizes at a much lower level. This illustrates that, in a static bandit, unnecessary exploration is detrimental to long-run reward.

In Figure 6 it is shown that in a stationary environment, lower ε (0.1) quickly converges to selecting the best arm most of the time, giving high optimal-arm percentages. Higher ε values (0.7, 0.9) keep exploring too often, so their optimal-arm selection rate remains low even after many steps. The Softmax policy rapidly concentrates the highest probability on the best arm, while still doing some graded exploration, so it ends up with the highest optimal-arm selection.

All methods increase cumulative reward but the slopes separate as the environment changes. The average estimator (green) initially performs well but its curve flattens compared to the others, reflecting slower adaptation and more pulls of suboptimal arms. Moving average (yellow) does better, and the incremental update (orange) achieves the steepest curve, the highest long-term cumulative reward, which means lower loss of potential in a non-stationary bandit.

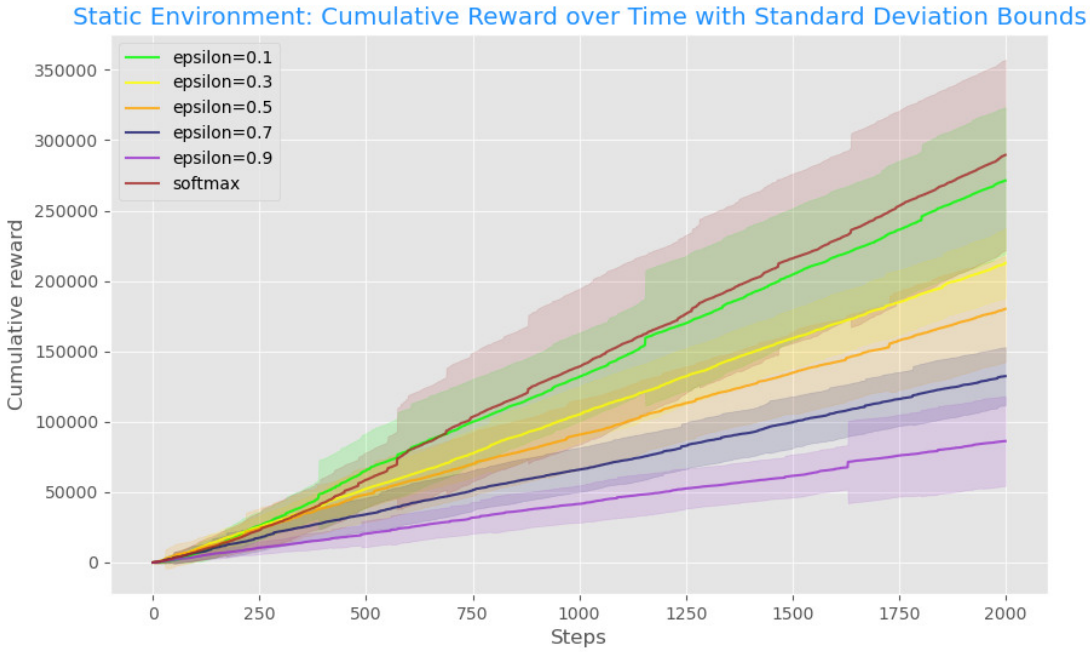

Figure 4: Cumulative reward over time with standard deviation bounds for static environment

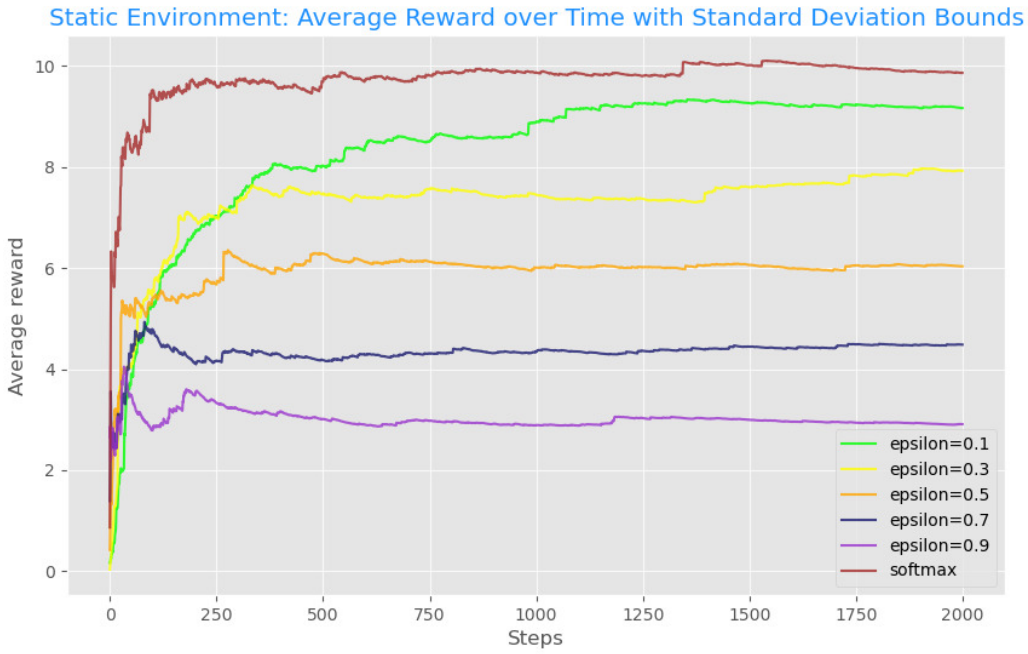

Figure 5: Average reward over time with standard deviation bounds for static environment.

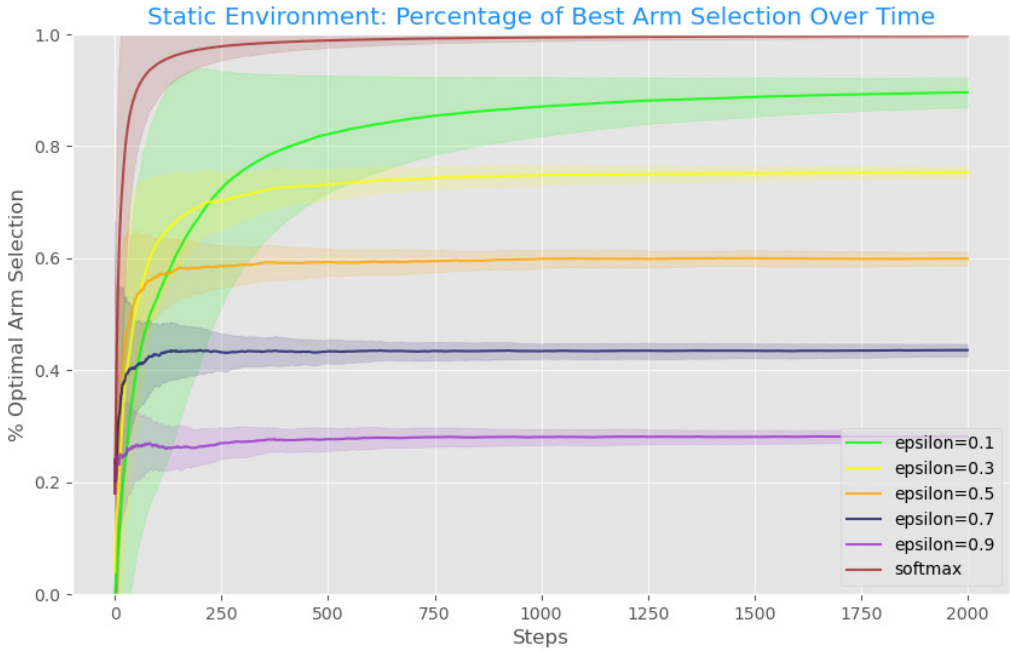

Figure 6: Percentage of best arm selection over time for static environment.

0.2 Dynamic Environment

At the start, all methods rarely select the best arm, because their value estimates are uncertain. The average estimator (green) learns reasonably fast but then degrades as the environment changes, since it keeps all past rewards and therefore reacts slowly to non-stationary bandits.

The moving average (yellow) reacts faster to changes than the plain average, so the percentage of best-arm selections improves more steadily. The incremental update with fixed step size (orange) adapts the quickest and maintains the highest fraction of optimal-arm selections over time, showing that constant-step updates are more appropriate in a dynamic environment.

Overall, these results support the observation that in static environments keeping exploration as low as possible is ideal. This is something that Softmax can easily account for. Whereas in dynamic environments it is necessary to implement estimators that adapt quickly to changes, which also includes Softmax.

- Experimental results show the average estimator decays over time because it calculates the value with historical memory
- Moving average shows lower efficiency at the start because it has lower information/data to work with, but over time it improves significantly due to the dynamic nature of the estimator
- For the incremental estimator, not only did it start off with relatively high optimal selection, but also it performed the most optimally over time. This is because it accumulates data results, and it has more accumulative data to work with.
- These results indicate that dynamic estimators are superior to the average estimator, and the incremental estimator is the most optimal (within our timespan). Given more time, there is a potential for the moving average to overtake the incremental estimator, but this cannot be accurately shown from the data.

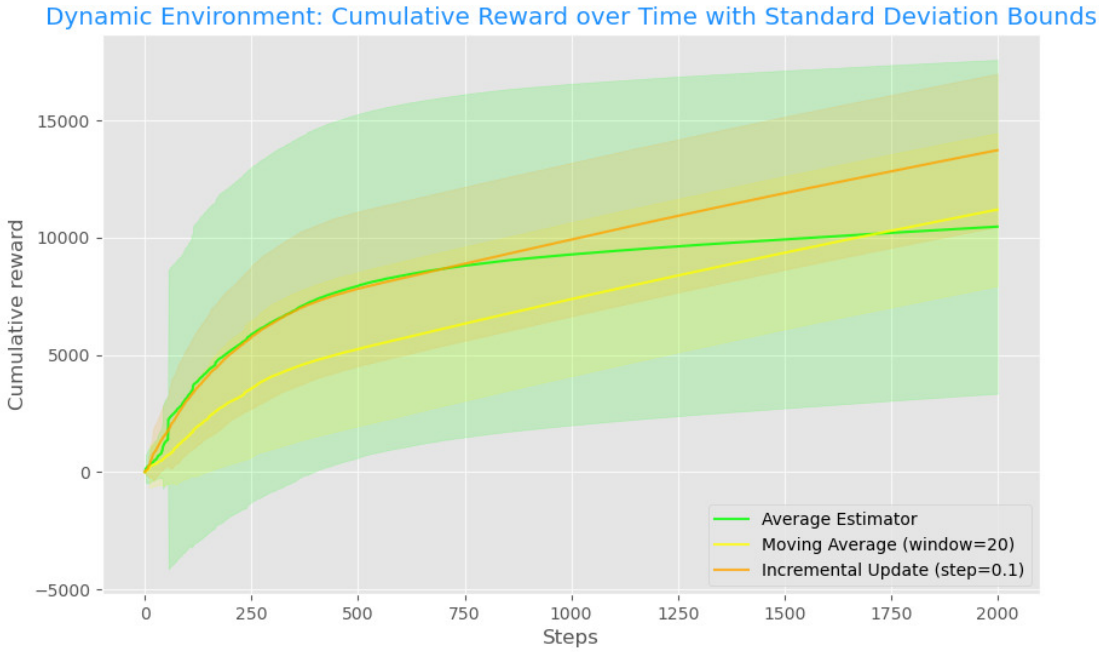

Figure 7: Cumulative reward over time with standard deviation bounds for dynamic environment

Discussion

Problem 2

The trade-off between exploration and exploitation is a central challenge in reinforcement learning. To obtain high reward, an agent should prefer actions that have produced good results in the past (exploitation), but to discover such actions it must also try actions that have not been tried often or at all (exploration). (Berger-Tal, Nathan, Meron, & Saltz, 2014). In other words, the agent must exploit what it already knows to gain immediate reward, while also exploring to improve its knowledge and make better action selections in the future. (Sutton & Barto, 2014)

In multi-armed bandits, the exploration-exploitation dilemma arises because, at each step, the agent must decide whether to play the arm that has produced the highest average reward so far (exploitation) or to try a different arm to improve its estimate of that arm's true value (exploration). Formally, each arm has an unknown reward distribution, and the agent only learns about these rewards through repeated interaction, which makes this trade-off unavoidable.

The goal of a multi-armed bandit agent is to find an appropriate balance between exploration and exploitation that minimises the regret over a given time horizon. Regret is defined as the difference between the cumulative reward that would have been obtained by always selecting the optimal arm and the cumulative reward obtained by the agent's strategy. A good bandit algorithm aims for regret that grows sublinearly with time, so that the average regret per step converges to zero. (Bubeck & Cesa-Bianchi, 2012)

One simple strategy to address this trade-off is the epsilon-greedy method. In epsilon-greedy, at each step the agent chooses a random arm with probability ε (exploration) and chooses the arm with the highest current estimated value with probability $1 - \varepsilon$ (exploitation). The parameter ε therefore controls the balance between exploration and exploitation: larger values of ε encourage more exploration, while smaller values favour exploiting the best-known arm. (Sutton & Barto, 2014)

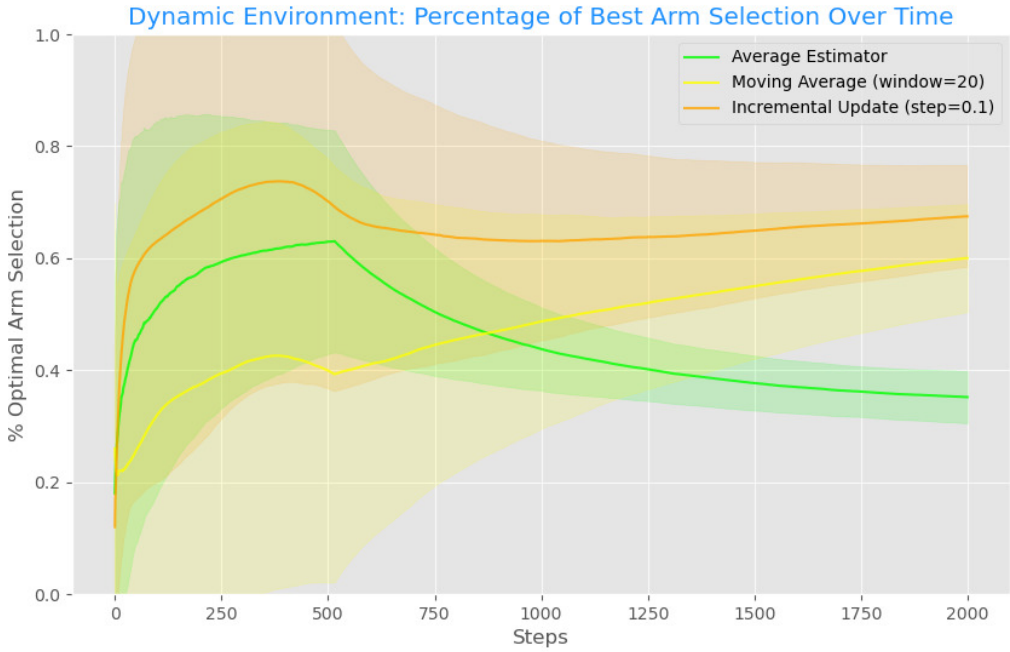

Figure 8: Percentage of Best Arm Selection over time in a Dynamic Environmentt

Problem 3

The true value of an action is the average reward that was received after that action was taken such that the difference between methods lies in how they use that uncertainty to balance exploration and exploitation.

In a more strict mathematical sense:

$$Q^{\pi}_{s,a} = E_{\pi}\{R_t | s_t = s, a_t = a\}$$

The action value with policy π, given the state space s and action space a, is equal to the estimated value of the reward R at the current time t given that the agent has taken action a_t under the state s_t.

Which can be calculated as:

$$E_{\pi}\{\sum_{k=0}^{\infty} \gamma^k r_{t+k+1} | s_t = s, a_t = a\}$$

This principle may be adapted based on the kind of estimator and policy of an agent.

Incremental Update Rule

It uses a fixed step size α, which makes recent rewards more important than old ones. This was the best estimator for the tested dynamic environment. It adapted the fastest to reward distribution changes and kept the highest percentage of optimal arm selections over time. It also started well from the beginning, not only at the end. The team notes that moving average could eventually overcome it given more time, but this was not confirmed by the data. Mathematicaly described with the expression:

$$Q_{n+1} = Q_n + \alpha[R_n - Q_n]$$

Sample-Average Estimator

This method estimates each arm's value by calculating the mean of all rewards received from that arm. In static environments it works well, especially when consistent with small epsilon values. The agent does not need much exploration once it learns a good approximation of the bandits' expected reward. However, in dynamic environments this estimator becomes a problem because it gives equal weight to old and new rewards, so it cannot adapt quickly with the environment changes. Formally defined as:

$$Q_n(a) = \frac{1}{n} \sum_{i=1}^{n} R_i$$

Moving Average Estimator

Instead of using all past single rewards, this method only considers the last N rewards (windows of 100 and 20 were used). At the beginning, it performs worse than the total average estimator because it has less data. But in dynamic environments, it improves over time, since it forgets old rewards and reacts better to changes. A window of 20 showed slightly better results than a window of 100 in most cases. Formally defined as:

$$Q_n(a) = \frac{\sum_{i=\max(1,n-W+1)}^{n} R_i}{\min(n,W)}$$

Where$Q_n(a) =_{\text{def}}$ the sum of the rewards from pull max $(1, n - W + 1)$to $\frac{n}{/min(n,W)}$

ε-Greedy Strategy

The agent exploits the best known arm with probability $1 - \varepsilon$ and explores randomly with probability ε where, as set for this study, $\varepsilon \in [0.1, 0.3, 0.5, 0.7, 0.9]$.

In static environments, low epsilon (0.1, -0.3) was clearly better because excessive exploration hurts performance when arm values do not change. In dynamic environments, moderate exploration was more useful because the agent needs to detect when the best arm changes. Initially, single runs were used with different random seeds. Results varied between seeds, which shows the stochastic nature of the problem. To account for this, the final experiments were performed in groups of 50 runs whose results were averaged. Looking at the conifidence intervals is a good to show the most likely space of single scenarios.

Softmax Strategy

Instead of a binary explore/exploit decision, Softmax assigns probabilities to each arm based on learned preferences, updated by gradient ascent. Thus, its function $\sigma(z)$ takes in a mathematical tuple made out of a k amount of estimations $(z_1 \dots z_k)$, and assigns them a value $\in (0, 1)$. I.E. $\mathbb{R}^K \to$ (0,1)^k; where $\sigma(z)$ is defined as:

$$\sigma(z)_i = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}$$

This implies that every input will be applied to the standard exponential function, and then normalised by the sum of every of such exponents.

The preference $-\beta$ for $\beta \in \mathbb{R}$ may be treated as a parameter that can be adjusted to either concentrate the parameters on the best estimations ($\beta > 1$), or spread them around lesser estimtions ($0 < \beta < 1$). This results in the equation:

$$\sigma(z)_i = \frac{e^{-\beta z_i}}{\sum_{j=1}^{K} e^{-\beta z_j}}$$

The code used for this study manages this parameter conceptually as a temperature, which but the reciprocal: $\beta = 1/T$.

Considerations on Action Value Methods

Softmax was the best strategy overall for static environments. It concentrated the probability on the best arm more raidly than epsilon-greedy, while still doing some exploration. And so, achieved the highest optimal-arm selection rate among all strategies tested.

In general, static environments need low exploration and stable estimators, while dynamic environments need methods that can forget old data and keep exploring. The incremental update is the most suitable estimator for non-stationary settings.

Conclusion

It was found that the Softmax strategy performed the most optimally within this particular setup; in contrast with Epsilon-greedy. The subsequent experiment tested for an optimal estimator with a group of Softmax policy agents. It was found that when faced with responsive bandits, the best estimator was incremental update, followed by moving average with a window of 20. The confidence intervals show that Incremental Update has a quicker response, and occasionally greatly outperforms the others. Also, the confidence intervals for the Moving Average estimator show that it can be less consistent and has less potential overall.

References

Antonacopoulos, A., Chaudhuri, S., Chellappa, R., Liu, C.-L., Bhattacharya, S., & Pal, U. (Eds.). (2025). *Pattern recognition: 27th international conference, ICPR 2024, kolkata, india, december 1–5, 2024, proceedings, part IX* (Vol. 15309). Springer Nature Switzerland. https://doi.org/10.1007/978-3-031-78189-6

Berger-Tal, O., Nathan, J., Meron, E., & Saltz, D. (2014). The exploration-exploitation dilemma: A multidisciplinary framework [Publisher: Public Library of Science]. *PLOS ONE*, *9*(4), e95693. https://doi.org/10.1371/journal.pone.0095693

Bubeck, S., & Cesa-Bianchi, N. (2012, November 3). Regret analysis of stochastic and nonstochastic multi-armed bandit problems. https://doi.org/10.48550/arXiv.1204.5721

Nandhini, S. K., & Pandian, J. A. (2025). Fractal dimension analysis for improved decision-making in non-stationary multi-arm bandit problems: A machine learning approach. *The European Physical Journal Special Topics*. https://doi.org/10.1140/epjs/s11734-025-01581-4

Sutton, R. S., & Barto, A. G. (n.d.). Reinforcement learning: An introduction.

Wang, H., Zariphopoulou, T., & Zhou, X. (2019, February 13). Exploration versus exploitation in reinforcement learning: A stochastic control approach. https://doi.org/10.48550/arXiv.1812.01552